Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System

CO3-ASSESSMENT 2

Course:

Software Engineering / Code Review and Version Control Evaluation

Name:

Davis Damien M

192521124

1. Problem Overview

1.1 Problem Statement

Hotels often manage reservations, room allocation, housekeeping, maintenance requests, guest communication, billing, and personalized services using multiple disconnected systems. This leads to inefficient operations, delayed service delivery, poor resource utilization, and inconsistent guest experiences.

The Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System aims to integrate hotel operations into a unified platform enhanced with AI-driven automation and decision support.

1.2 Implemented Solution

The reviewed project provides a centralized web-based hotel management platform with AI-assisted features such as:

- Intelligent room recommendation
- Automated guest service requests
- Housekeeping task optimization
- Predictive maintenance alerts
- Occupancy and revenue analytics
- AI chatbot for guest assistance
- Personalized guest experience management

The system uses a client-server architecture with a web frontend, backend API, database layer, and AI modules for analytics and automation.

1.3 Project Objectives

- Improve operational efficiency
- Enhance guest satisfaction
- Automate repetitive hotel workflows
- Provide real-time analytics and reporting

- Enable intelligent decision-making using AI

1.4 Key Functionalities

Module	Functionality
Reservation	Booking, cancellation, modification
Room Management	Availability, allocation, room status
Housekeeping	Task assignment and tracking
Guest Services	Requests, complaints, concierge support
Billing	Invoice generation and payment tracking
AI Engine	Recommendations, chatbot, predictive analytics
Dashboard	Occupancy, revenue, service metrics

2. Repository Organization

2.1 Repository Structure

Example repository layout:

```
project-root/
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── services/
│   ├── middleware/
│   └── app.py
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── assets/
```

```
│                   └── package.json
├── ai/
│   ├── recommendation.py
│   ├── occupancy_predictor.py
│   └── chatbot.py
├── database/
│   ├── schema.sql
│   └── seed_data.sql
├── tests/
├── docs/
├── screenshots/
├── README.md
├── requirements.txt
└── .gitignore
```

2.2 Evaluation

Strengths

- Clear separation of frontend, backend, AI, and database components.
- Modular folder organization.
- Logical grouping of source files.
- Independent testing directory.
- Documentation stored separately.

Weaknesses

- Some utility functions are duplicated.
- Configuration files are scattered.
- AI scripts could be packaged as a reusable module.

2.3 Maintainability

The repository supports maintainability by:

- Isolating business logic from presentation logic.

- Organizing routes and controllers separately.
- Keeping AI models independent from web components.
- Providing a dedicated tests directory.

This structure enables multiple developers to work simultaneously with minimal merge conflicts.

3. Code Quality Review

3.1 Readability

Strengths

- Meaningful variable names
- Descriptive function names
- Consistent indentation
- Appropriate use of comments
- Modular functions

Example:

```
def recommend_room(guest_profile):
available_rooms = get_available_rooms()
return rank_rooms(available_rooms, guest_profile)
```

The function name clearly expresses its purpose.

3.2 Modularity

The application follows modular design principles.

Example modules:

- reservation_service.py
- housekeeping_service.py
- billing_service.py
- ai_recommendation.py

Advantages:

- Reusability
- Easier debugging
- Independent testing
- Better scalability

3.3 Coding Standards

The code largely follows standard practices:

- PEP 8 (Python)
- Consistent naming conventions
- CamelCase for classes
- snake_case for variables and functions

Areas for Improvement

- Some functions exceed 50–80 lines.
- Hardcoded configuration values appear in source files.
- Exception handling is inconsistent.

Example improvement:

Current:

```
price = room.price * 1.18
```

Improved:

```
GST_RATE = 1.18
price = room.price * GST_RATE
```

3.4 Documentation

Most public functions contain docstrings.

Example:

```
def assign_housekeeping(room_id):
```